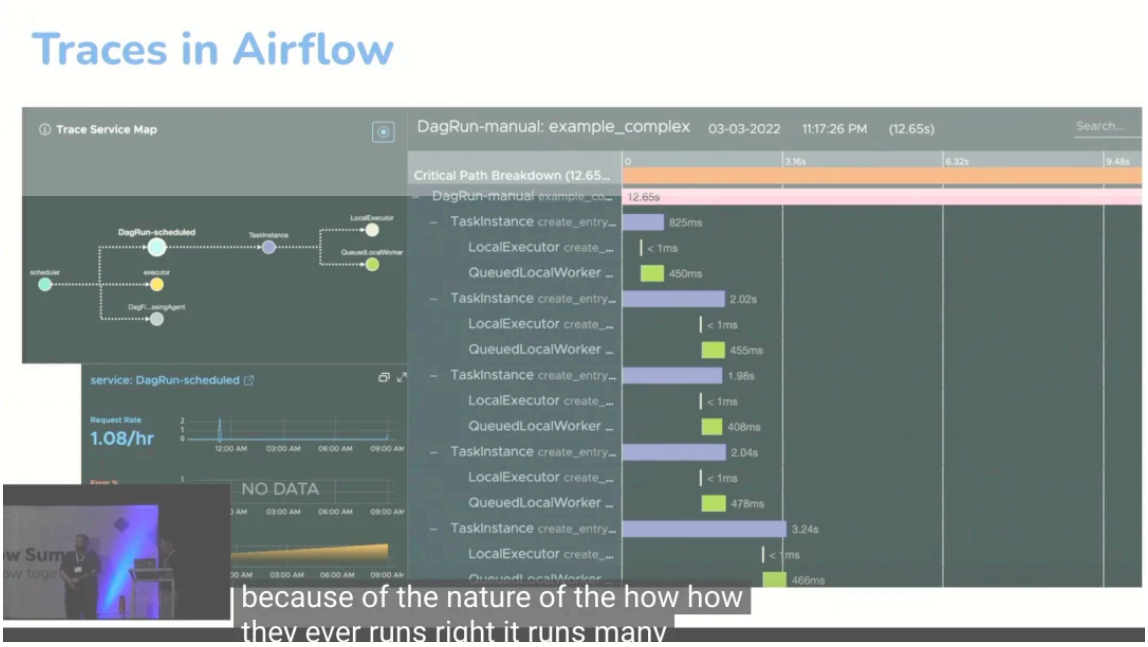
OpenTelemetry实现更好的Airflow可观测性

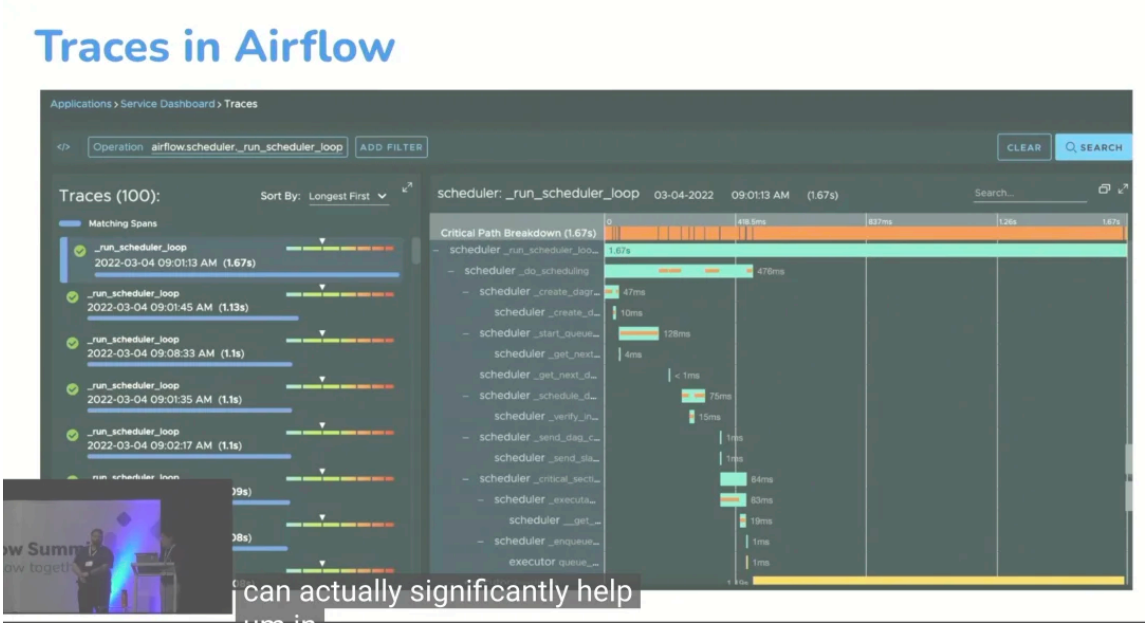
DevOps云学堂 2023年10月04日 00:54

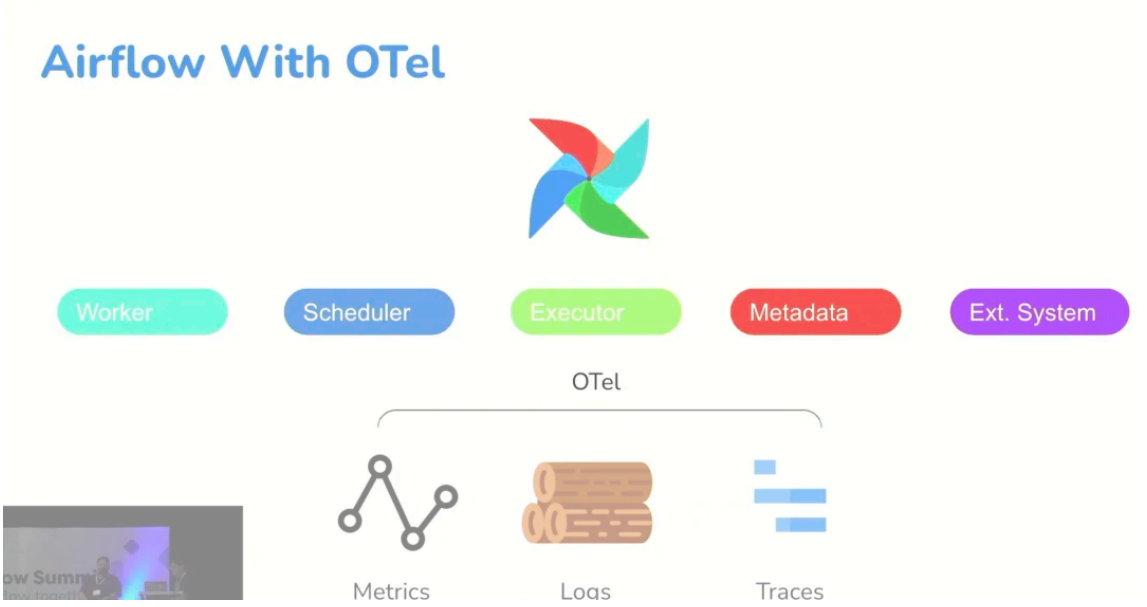
本篇文章是「DevOps云学堂」与你共同进步的第 56 篇

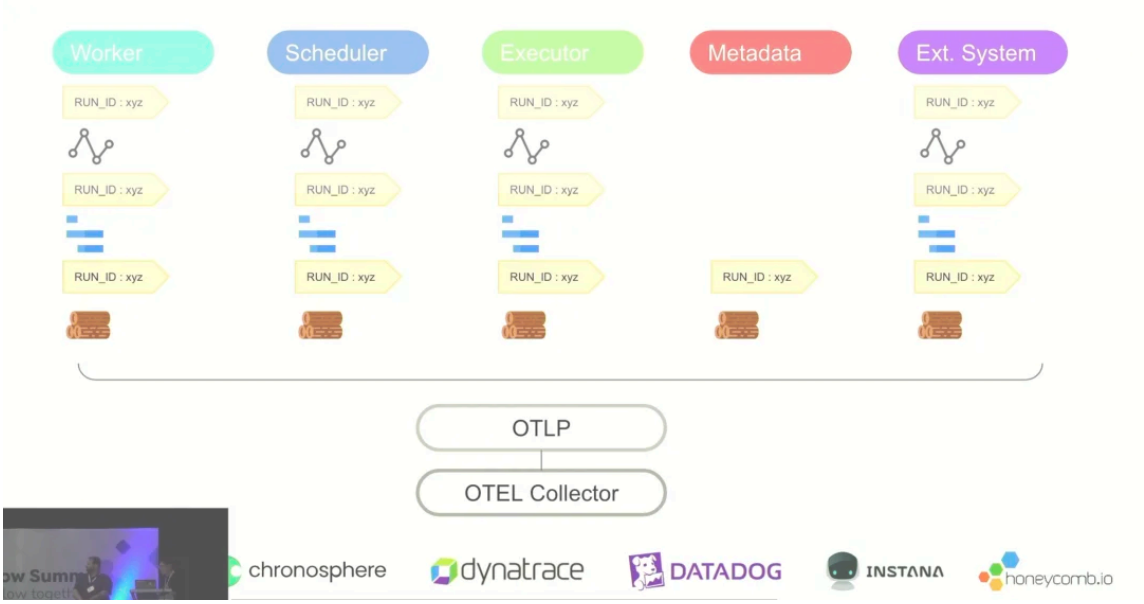
> 分享来源于Airflow Summit 2023 https://youtu.be/XSzJSRgWLfY?feature=shared



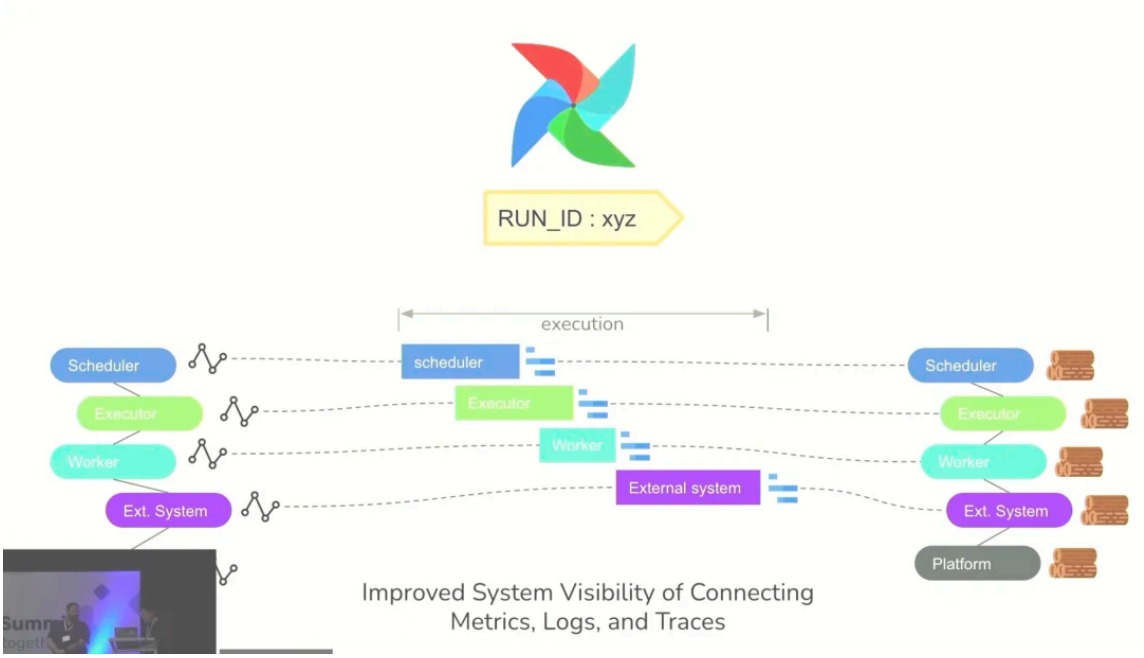

Traces in Airflow
Trace Service Map
DagRun-manual: example_complex 03-03-2022 11:17:26 PM (12.65s)
Search...
Critical Path Breakdown (12.65...
DagRun-manual example_co... 12.65s
TaskInstance create_entry... 825ms
LocalExecutor create_... < 1ms
QueuedLocalWorker ... 450ms
TaskInstance create_entry... 2.02s
LocalExecutor create_... < 1ms
QueuedLocalWorker ... 455ms
TaskInstance create_entry... 1.98s
LocalExecutor create_... < 1ms
QueuedLocalWorker ... 408ms
TaskInstance create_entry... 2.04s
LocalExecutor create_... < 1ms
QueuedLocalWorker ... 478ms
TaskInstance create_entry... 3.24s
LocalExecutor create_... < 1ms
QueuedLocalWorker ... 466ms
service: DagRun-scheduled
Request Rate
1.08/hr
NO DATA
because of the nature of the how how
they ever runs right it runs many


Traces in Airflow
Applications > Service Dashboard > Traces
Operation airflow.scheduler._run_scheduler_loop
ADD FILTER
CLEAR
SEARCH
Traces (100):
Sort By: Longest First
Matching Spans
_run_scheduler_loop 2022-03-04 09:01:13 AM (1.67s)
_run_scheduler_loop 2022-03-04 09:01:45 AM (1.13s)
_run_scheduler_loop 2022-03-04 09:08:33 AM (1.1s)
_run_scheduler_loop 2022-03-04 09:01:35 AM (1.1s)
_run_scheduler_loop 2022-03-04 09:02:17 AM (1.1s)
scheduler: _run_scheduler_loop 03-04-2022 09:01:13 AM (1.67s)
Search...
Critical Path Breakdown (1.67s)
scheduler _run_scheduler_loo... 1.67s
scheduler _do_scheduling 478ms
scheduler _create_dagr... 47ms
scheduler _create_d... 10ms
scheduler _start_queue... 128ms
scheduler _get_next... 4ms
scheduler _get_next_d... < 1ms
scheduler _schedule_d... 75ms
scheduler _verify_in... 15ms
scheduler _send_dag_c... 1ms
scheduler _send_sla... 1ms
scheduler _critical_secti... 84ms
scheduler _executa... 83ms
scheduler _get_... 19ms
scheduler _enqueue... 1ms
executor queue_... 1ms
can actually significantly help


Airflow With OTel
Worker
Scheduler
Executor
Metadata
Ext. System
OTel
Metrics
Logs
Traces



Apache Airflow是一个编排平台，用于以编程方式编写、安排和执行工作流。OpenTelemetry开放遥测用于生成、收集和导出遥测数据（指标、日志和跟踪），以帮助您分析软件的性能和行为。这两个开源项目看起来很自然，随着 Airflow 2.7 的推出，用户现在可以开始在 Airflow 中利用 OpenTelemetry Metrics！



Airflow 支持通过 StatsD 发出指标已经有一段时间了，并且一直可以通过标准 python 记录器进行日志记录。完整的 OpenTelemetry 集成将使这两个功能合并到一个开源标准中，同时还添加跟踪。OpenTelemetry Traces 可以更好地了解管道如何实时执行以及各个模块如何交互。虽然下一步是整合计划，但目前还没有确定的日期。

配置您的Airflow环境

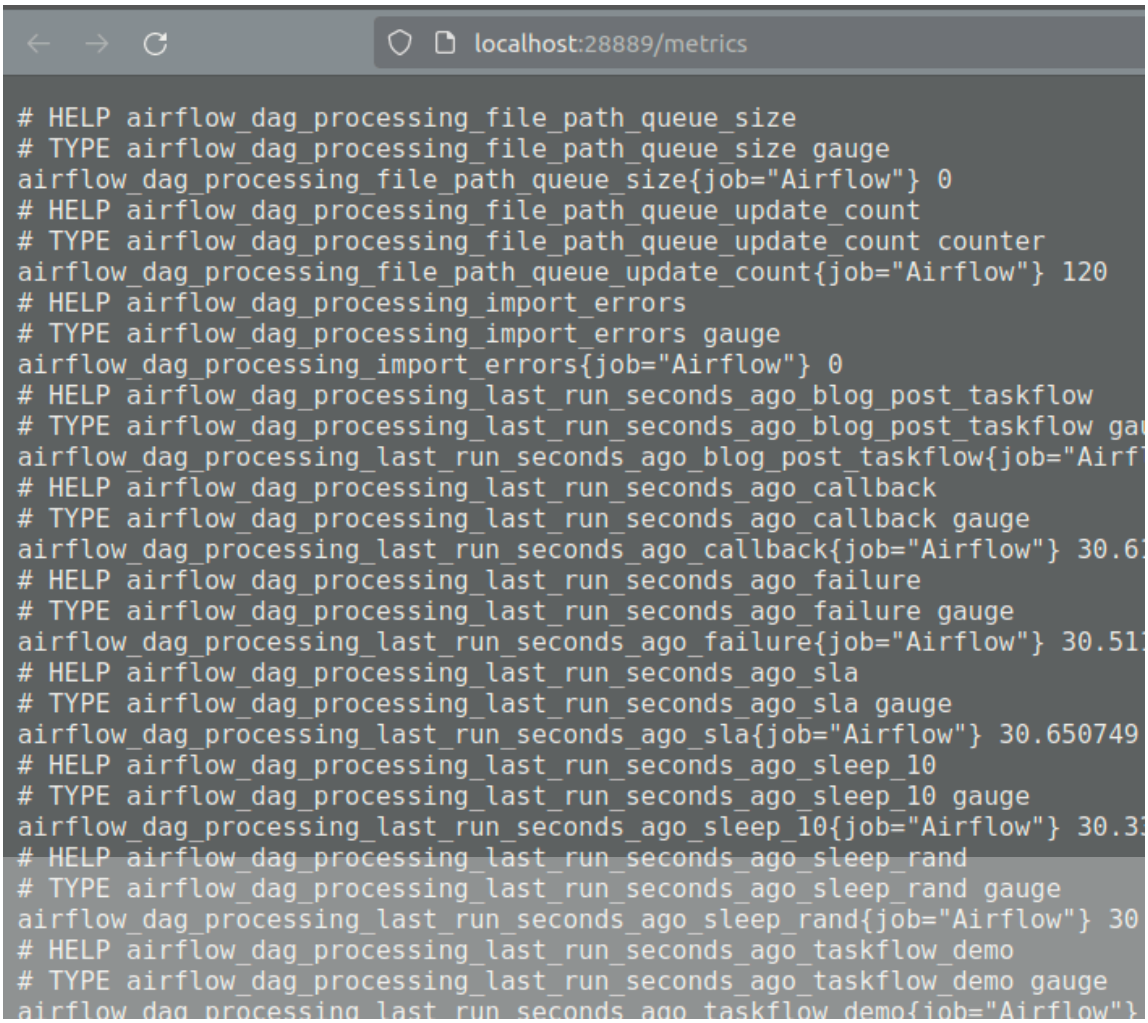
要在现有 Airflow 环境中启用 OpenTelemetry，您需要安装 `otel` 附加包并配置几个环境变量，如Airflow 文档页面中所述。您还需要配置otel-collector和某种形式的可观察平台来可视化和监控这些指标。在这篇文章中，我将使用Prometheus作为指标后端来存储数据，并在Grafana中构建一个仪表板来可视化它们。

OTel收集器

OpenTelemetry Collector 提供了关于如何接收、处理和导出遥测数据的与供应商无关的实现。收集器会将所有 Airflow 指标收集到 Prometheus 获取它们的中心位置。

如需配置帮助，请参阅OpenTelemetry Collector 入门指南，并查看与 Airflow 开发环境（称为 Breeze）捆绑在一起的Docker Compose 文件和otel-collector 配置文件。

如果您使用了上面 Airflow 页面中的设置，并且让 Airflow 和您的 OTel Collector 在本地 Docker 容器中运行，您可以将浏览器指向localhost:28889/metrics。在这里您将看到 Prometheus 格式的所有可用原始指标。它们看起来像这样：



otel-collector 显示的原始指标数据

我们在看什么？

那是一面文字墙，我们实际上在这里看到的是什么？发出的每个指标在此页面上都有三行：

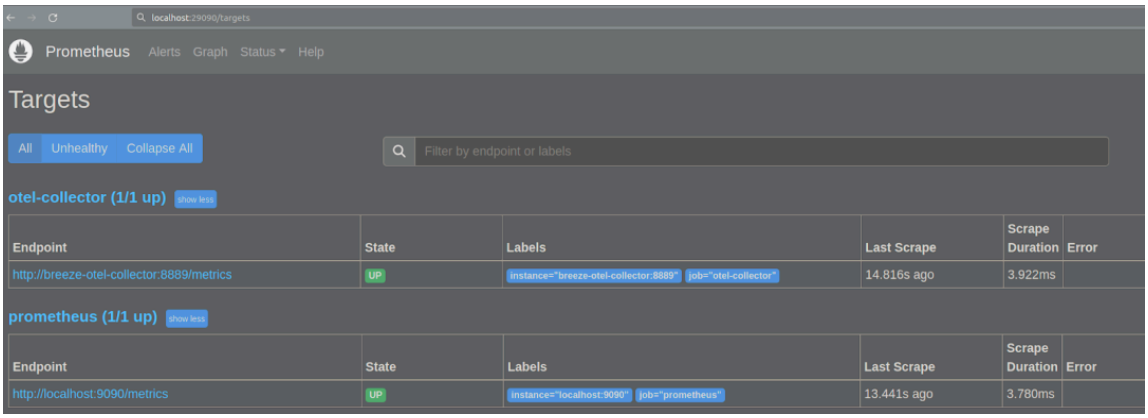
1. HELP 尚未实现，但最终将包含指标的描述。
2. TYPE 将为“计数器”、“仪表”或“计时器”之一。如果这些术语对您来说是新的，也许可以跳到附录 1 获取非常简短的总结。
3. 第三行是指标的名称、任何适用的标签以及当前值。

您可以想象直接提取并解析这些数据，但现有的解决方案可以做到这一点。我们将看看下面的一个选项。

Prometheus

Prometheus将成为我们的监控和存储解决方案。他们的网站有一个很好的入门指南。Breeze Docker Compose 文件（上面链接）和Prometheus 配置文件对于入门也可能很有用。

如果您已使用推荐的配置成功启动指标页面，您应该能够在localhost:29090/targets处查看目标并看到如下内容：

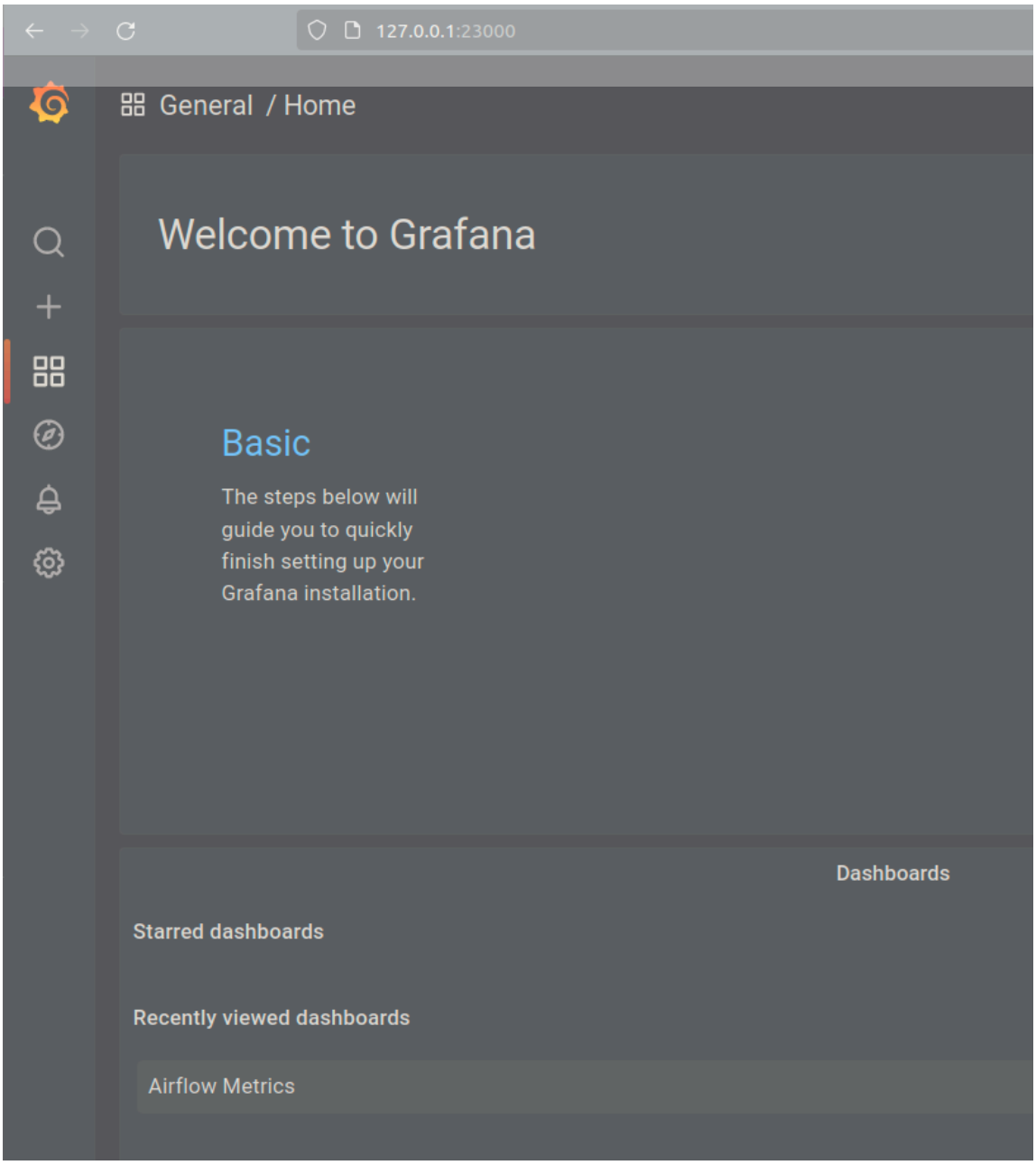


Prometheus 中的 `Targets` 页面显示与 `otel-collector` 的活动连接

Grafana

最后一步是设置Grafana。借助 Grafana，您可以通过美观、灵活的仪表板创建、探索和共享所有数据。他们提供付费托管服务，但为了演示，您可以在另一个 Docker 容器中使用他们的免费开源版本。Breeze Docker Compose 文件（上面链接）和Breeze 配置文件可以帮助您进行设置。请注意，对于 Grafana，配置文件分布在几个目录中，并包含用于配置数据源和简单的默认仪表板的文件。

如果一切都使用建议的设置运行，您可以将浏览器指向localhost:23000并查看您的 Grafana 登录页面！



默认 Grafana 登陆页面

制作您的第一个 Grafana 仪表板

如果您已经走到这一步，那么恭喜您！您可以使用 Airflow 来使用完整的可观察性堆栈！现在，让我们来看看。

在您探索 Grafana 之前，下面是一个示例演示 DAG，它每分钟运行一次并执行一项任务，即等待 1 到 10 秒之间的随机时间长度。将其放入 DAG 文件夹中，启用它，并让它运行多个周期，

以在您浏览时生成一些指标数据。我们稍后将使用它生成的数据，它运行的时间越长，它看起来就越好。因此，请放心让它运行并离开一段时间，然后再继续。

```
import time

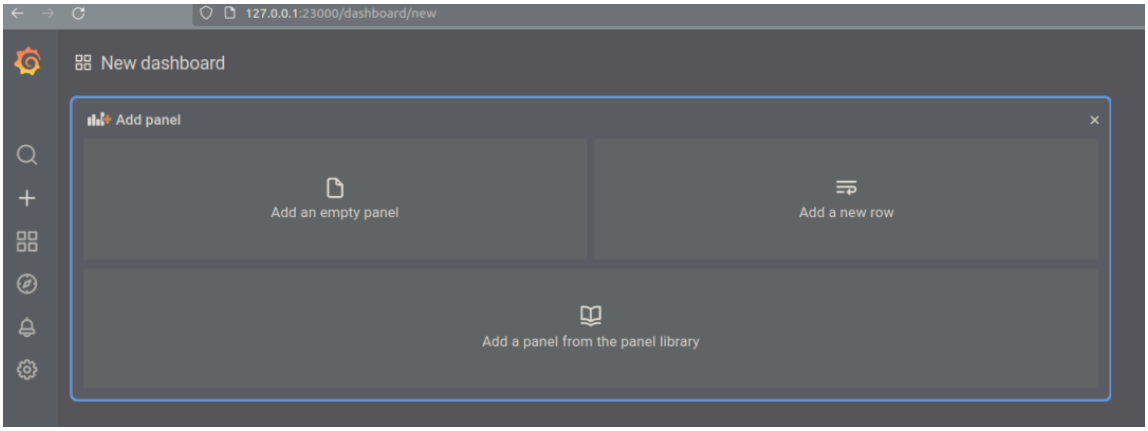
from airflow import DAG
from airflow.decorators import task
from airflow.utils.timezone import datetime
from datetime import timedelta
from random import randint


@task
def task1():
    time.sleep(randint(1, 10))

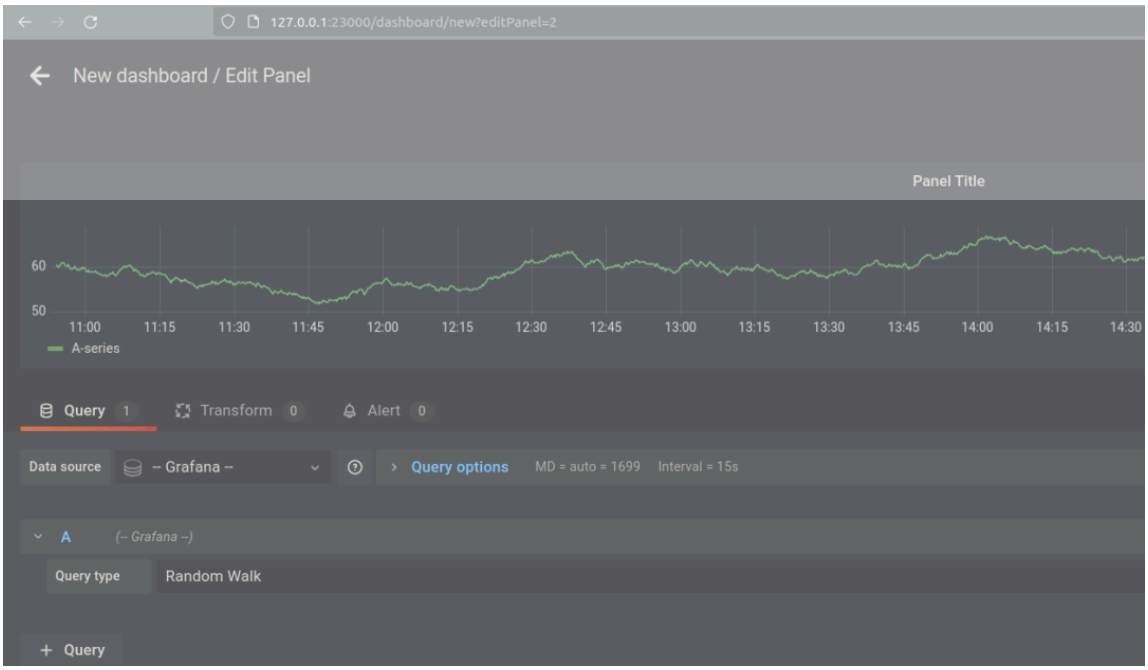

with DAG(
    dag_id='sleep_random',
    start_date=datetime(2021, 1, 1),
    schedule_interval=timedelta(minutes=1),
    catchup=False
) as dag:

    task1()
```

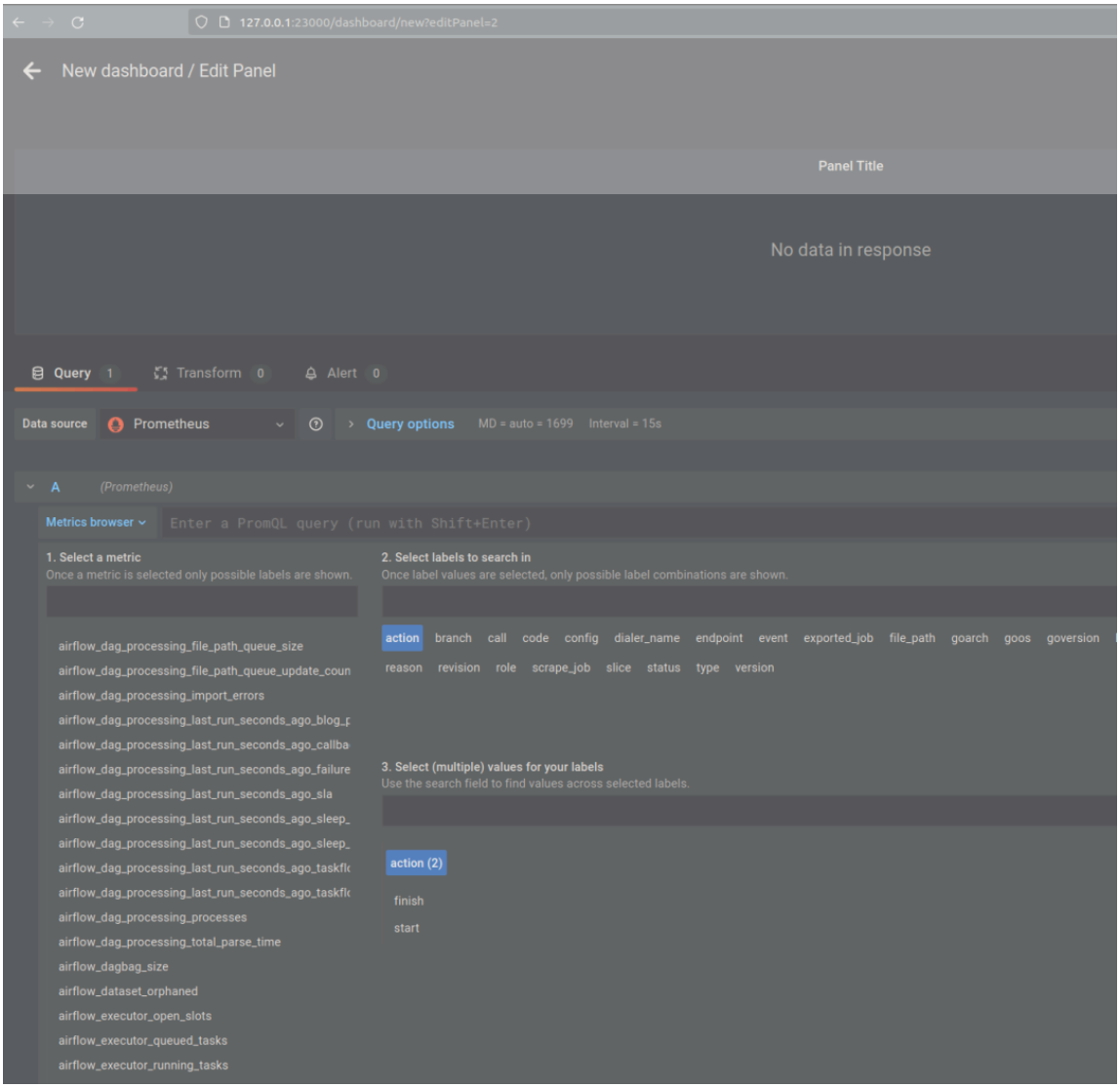
运行一段时间后：切换到 Grafana，创建一个新的仪表板（最左侧的加号），然后在该新仪表板中添加一个新的空面板。



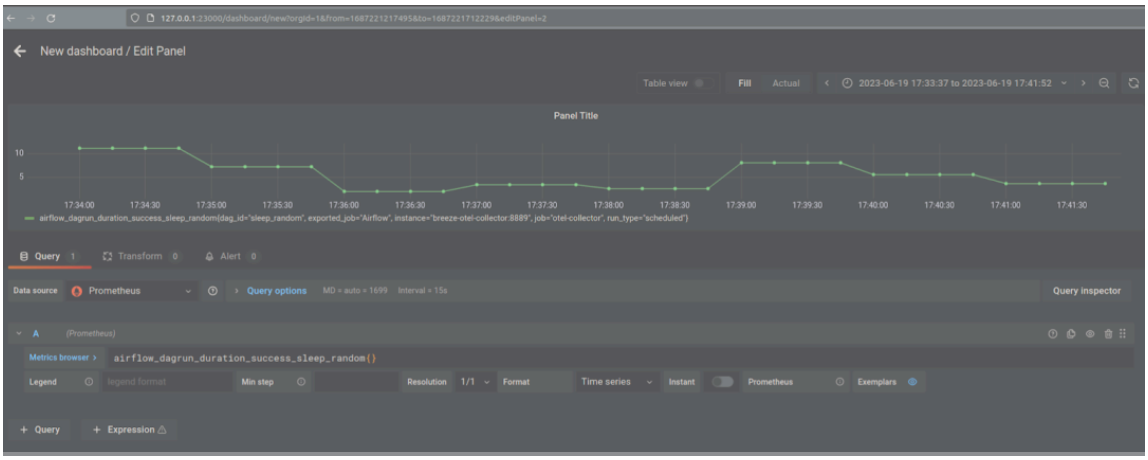
默认情况下，您会看到一个漂亮的随机游走图：



将数据源更改为 Prometheus ，然后单击新的 Metrics Browser 按钮。这将为您提供所有可用指标的列表。花一点时间看看可用的内容。如果您最近运行过任何 DAG，将会有各种关于 任务运行计数 和 持续时间 、 成功计数 等的可用指标。如果您没有运行任何 DAG，您仍然会看到一些选项，例如 dagbag 大小 、 调度程序心跳 和 其他系统指标 。根据您的系统，可能还存在大量我们在本文中不一定关心的其他问题。默认情况下，Airflow 发出的所有指标都以 airflow_ 为前缀，因此按此过滤可以帮助缩小选择范围。

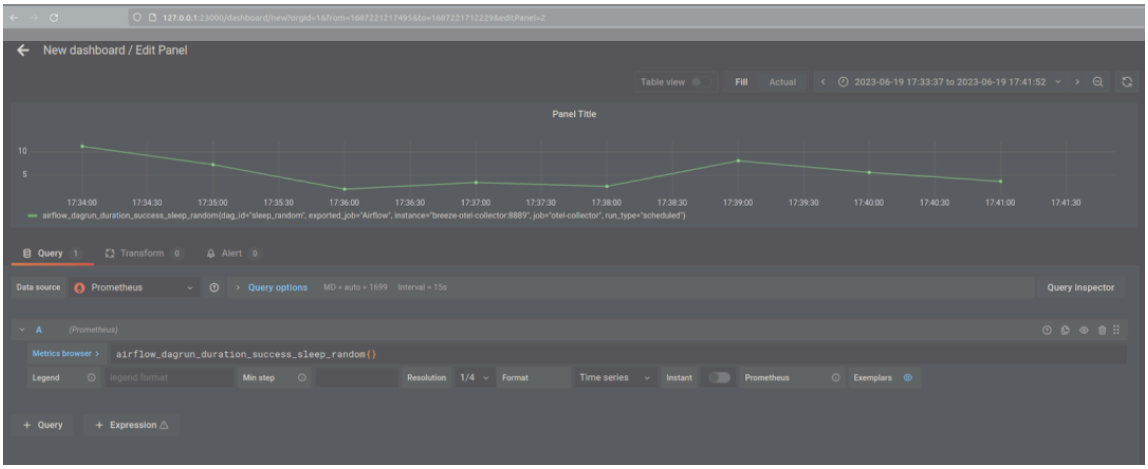


如果您给 DAG 半小时左右的时间来构建一些指标，请使用指标浏览器查找名为 `airflow_dagrun_duration_success_sleep_random` 的指标。将其他字段保留为默认设置，然后单击 使用查询 。你应该可以看到这样的图表：

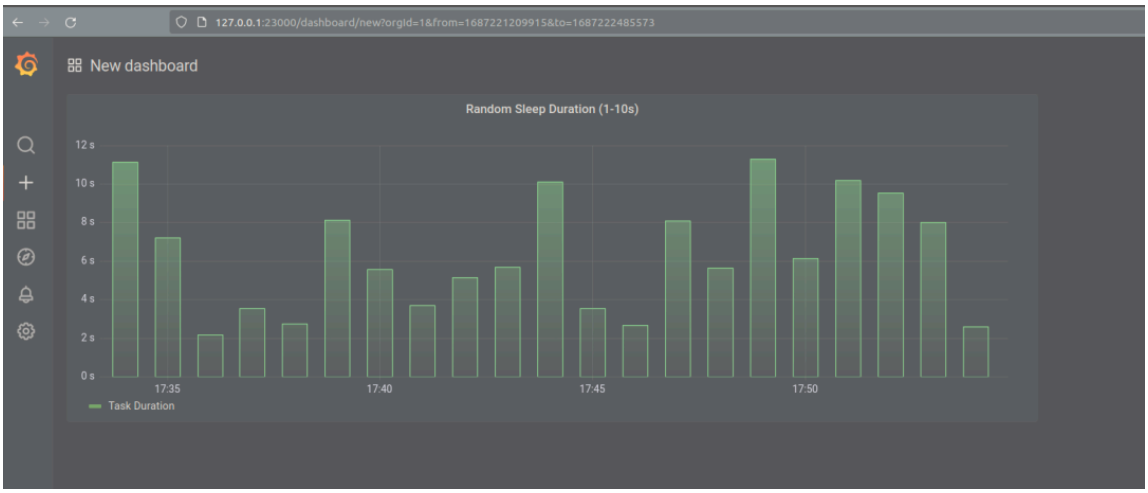


为您的查询起一个好听的名称，例如 图例 字段中的 任务持续时间 。根据您的配置值，您可能希望调整 分辨率 ，以便我们显示每个第 N 个值。如果您看到相同的值每次重复四次，如上面的

屏幕截图所示，您可以将分辨率调整为 1/4 ，也可以调整 OTEL_INTERVAL 环境值 （然后重新启动 Airflow 并重新运行 DAG 并等待值再次生成） 。将分辨率设置为 1/4，您将看到更清晰的图表：



现在我们可以使用可能折叠的右侧菜单选项卡。如果您没有看到右侧的选项，右上角 应用 按钮正下方有一个箭头可以显示它。给你的面板命名，例如 随机睡眠持续时间（1-10秒），也许将其设置为填充不透明度为50的条形图，并将渐变模式设置为 不透明度 。在 标准选项 下，我们可以将单位设置为 时间/秒（s），将最小值设置为0，最大值设置为12。玩完后，单击右上角的“应用”。这将使您返回仪表板视图，您应该看到类似这样的内容！



这里有一个图表，显示每次运行该 DAG 所需的时间。您会记得我们告诉它等待 1 到 10 秒之间的随机时间长度，因此它看起来应该非常随机。您可能还会注意到，有些时间略长于 10 秒。这是由于系统开销造成的，这正是您可能希望使用这些指标的原因之一！虽然该任务实际上休眠了长达 10 秒，但在启动和结束所附加的任务时会产生一些系统开销。

在上图中，我们可以看到总开销始终低于 2 秒，因为图表从未达到 12 秒。仔细观察实际指标数字可以发现，开销平均约为 1.2 秒，而且我认为这对于我的用例来说是可以接受的。如果这是生产环境，

将该面板向任一方向拖动得更大，请注意 Grafana 将自动调整两个轴上的比例和标签！当您找到喜欢的尺寸时，单击右上角的刷新按钮（在 Grafana 中，不适用于浏览器选项卡！），然后选择一个频率以使其自动更新。您现在应该有一个仪表板，它显示您的任务持续时间，并在 DAG 运行时每分钟左右自动更新为新值！

下一步是什么?

你接下来要做什么?

如果您有兴趣探索更多有关如何更好地利用 Grafana 构建更好的仪表板和警报的信息，Grafana 基础知识(https://grafana.com/tutorials/grafana-fundamentals/)指南可能是一个不错的起点。如果您有兴趣了解有关 Airflow 的更多信息或有任何疑问，请加入Airflow 社区 slack 服务器上的对话！https://apache-airflow-slack.herokuapp.com/

Airflow 和 OpenTelemetry 的下一步是什么?

接下来，我们将添加对 OTel 最有趣的功能的支持：跟踪！跟踪让我们了解管道运行时幕后实际发生的情况，并有助于可视化其任务运行的完整“路径”。例如，当与我们已经探索过的持续时间指标相结合时，我们将能够自动生成甘特图，以帮助找到减慢 DAG 速度的瓶颈。

附录 1 — 指标的简要概述

目前 Airflow 支持三种类型的指标：计数器、仪表和计时器。本附录将非常简短地概述这些在 Airflow 中的含义。

Counters

计数器是按值递增或递减的整数。截至撰写本文时，除了一个之外，所有计数器都是单调计数器，这意味着它只能增加。例如，您汽车中的里程表或自您启动 Airflow 以来完成的任务数。如果你可以说“再加一个”，那么你很可能正在处理一个计数器。请参阅 https://airflow.apache.org/docs/apache-airflow/stable/administration-and-deployment/logging-monitoring/metrics.html#counters以获取 Airflow 中可用的计数器列表。

Gauges

仪表是可以上升或下降的浮子。计数器和仪表之间的主要区别在于，仪表是瞬时读数，而不是增量变化。例如，考虑一下您的温度计或行李包中的 DAG 数量。当您读取温度计时，您会看到当前温度，但通常不会看到“它比您上次查看时高了三度”。如果您发现自己在想“当前价值是多少？”您可能正在考虑一个仪表。请参阅https://airflow.apache.org/docs/apache-airflow/stable/administration-and-deployment/logging-monitoring/metrics.html#gauges，了解气流中可用的仪表列表。

Timers

定时器是最明显的类型。正如您所期望的，他们报告了某件事花费了多长时间。请参阅 https://airflow.apache.org/docs/apache-airflow/stable/administration-and-deployment/logging-monitoring/metrics.html#timers以获取 Airflow 中可用的计时器列表。

深入研究

有关 Airflow 中指标的更多信息，请查看https://airflow.apache.org/docs/apache-airflow/stable/administration-and-deployment/logging-monitoring/metrics.html#metric-descriptions文档。有关 OpenTelemetry 支持的指标类型的更多信息，请参阅此处的 https://opentelemetry.io/docs/concepts/signals/metrics/文档。

持续输出 68 DataOps 6 Airflow 8

持续输出 · 目录

上一篇
Terraform 预提交挂钩使用指南：节省时间并提高代码质量

下一篇
Kubernetes–Pod的重新平衡和碎片整理